

PROMPT IA 08

Platform Engineering - Kubernetes Production

Multi-site + GitOps + Security + Observability + Backup/PRA + Developer Platform

Mission : industrialiser la plateforme OSS qui heberge les vertical slices deja developpes, avec deux sites autonomes, securite zero-trust, GitOps, observabilite et restauration testee.

100 % OSS/self-hosted first - automation first - managed-ready

1. Role de l'IA

Tu es Principal Platform Engineer / SRE. Tu dois produire une plateforme Kubernetes de production reproductible depuis Git. Ne fais aucune modification manuelle permanente : toute configuration durable doit être déclarée en IaC/GitOps lorsqu'elle peut raisonnablement l'être.

Ne construis pas tout en une seule fois. Livre un site fonctionnel minimal, teste-le, factorise, puis reproduis le second site. Toute action de production doit être auditable.

2. Architecture cible

```
GLOBAL
PowerDNS + dnsmist
|
+-----+-----+
| |
SITE A SITE B
| |
HAProxy HAProxy
| |
Caddy + Coraza Caddy + Coraza
| |
Kong Kong
| |
Istio Gateway Istio Gateway
| |
Kubernetes A Kubernetes B

Chaque site:
- 3 control-plane
- 5 workers cibles
- Cilium + Hubble
- Istio Ambient
- FluxCD
- OpenBao
- CloudNativePG
- Strimzi Kafka
- RabbitMQ
- Redis Cluster
- OpenSearch
- Rook-Ceph / LocalPV
- Prometheus/Grafana/Loki/Tempo
```

3. Provisioning et bootstrap

Utiliser Terraform pour l'infrastructure, Ansible/AWX pour OS, hardening et bootstrap, et Cluster API lorsque le provider cible est suffisamment mature.

```
Terraform
-> servers/network/DNS prerequisites
Ansible + AWX
-> Ubuntu hardening/bootstrap
Cluster API / kubeadm path
-> Kubernetes
FluxCD
-> platform components
```

Le fournisseur peut livrer Ubuntu préinstallé. Le rôle Ansible doit converger cet OS vers la baseline voulue au lieu d'exiger une installation manuelle minimale.

4. Git Governance

Mettre en place branches protégées, CODEOWNERS, signatures, reviews obligatoires, double approbation Tier 0, credentials CI minimaux, audit et sauvegarde Git immuable avec restauration testée.

```
change -> branch -> PR -> CI -> policy checks -> approvals -> merge
-> FluxCD reconciliation -> cluster
```

Aucun kubectl apply direct en production hors procedure break-glass documentee.

5. Kubernetes baseline

Configurer namespaces, ResourceQuota, LimitRange, requests/limits obligatoires et PriorityClass. Les workloads critiques ont PDB, anti-affinity et topologySpreadConstraints.

Pod Security restricted par default : runAsNonRoot, readOnlyRootFilesystem, allowPrivilegeEscalation=false, drop ALL capabilities, seccomp RuntimeDefault.

6. Reseau et service mesh

Cilium est le CNI officiel avec default-deny NetworkPolicies et Hubble. Istio Ambient fournit mTLS/L4 via ztunnel et waypoint uniquement quand une politique L7 est necessaire.

```
Pod -> Cilium
-> Istio Ambient
-> Service

Internet egress:
Pod -> Istio Egress Gateway -> Squid -> allowlist -> Internet
```

Pas d'accès Internet direct depuis les pods applicatifs. Squid est HA, journalise et sans interception TLS generale.

7. Edge et load balancing

Niveau	Composant	Role
Global	PowerDNS + dnsmist	GSLB, health/failover Site A/B.
API/site	HAProxy	Load balancing entree API.
Web/CDN	Apache Traffic Server	Cache/reverse proxy vers Next.js/assets.
API governance	Kong	Auth, rate limiting, routes, webhooks.
Mesh	Istio Ambient	Traffic policy, canary, mTLS.
Interne	Cilium/K8s Service	Load balancing Pods/Services.

8. Secrets, identites et chiffrement

Humains via Keycloak/OIDC, workloads via SPIFFE/SPIRE, secrets et credentials dynamiques via OpenBao. External Secrets Operator seulement lorsqu'un composant exige un Kubernetes Secret.

Activer Kubernetes encryption-at-rest avec KMS relie a OpenBao/HSM et rotation automatisee des cles etcd. Aucun secret applicatif longue duree dans Git.

9. Policy-as-Code

OPA est le PDP central de la Platform API avec decisions ALLOW/APPROVAL/DENY. Kyverno controle l'admission et la conformite Kubernetes.

```
Developer intent -> Platform API -> OPA
+--> ALLOW
+--> APPROVAL
+--> DENY

Manifest -> Kubernetes admission -> Kyverno -> accept/reject
```

10. Stateful platform

Deployer CloudNativePG, Strimzi Kafka, RabbitMQ Cluster Operator, Redis Cluster et OpenSearch selon les standards HA du projet.

Kafka : 3 brokers/site, LocalPV NVMe, RF=3, min ISR=2. RabbitMQ : 3 noeuds/site, Quorum Queues. PostgreSQL critique : LocalPV NVMe et endpoints writer/readers via PgBouncer.

Kafka inter-site : cluster independant par site + MirrorMaker 2. Pas de cluster Kafka ou RabbitMQ etendu sur WAN.

11. Stockage

```
nvme-local -> PostgreSQL critique / Kafka / workloads IOPS
ceph-rbd -> volumes bloc RWO generalistes
cephfs -> RWX seulement si necessaire
object -> stockage objet OSS self-hosted
```

Un cluster Ceph par site, jamais de Ceph etendu sur WAN. Replication n'est pas backup.

12. Observabilite

Prometheus + Alertmanager pour metrics/alertes, Grafana pour dashboards, Loki pour logs, Tempo pour traces, Hubble pour reseau, Thanos pour vue multi-site et long terme.

```
Application -> OpenTelemetry
+> metrics -> Prometheus -> Thanos
+> traces -> Tempo
+> logs -> Loki
```

Security/audit ciblé -> Splunk

Alerting oriente SLO, symptomes utilisateurs, metier et securite. Eviter les alertes sur simples pics CPU sans impact.

13. SLO et capacity

Configurer les SLO existants : API publique 99,9 %, Order/Payment 99,95 %, REST p95 < 300 ms, gRPC p95 < 100 ms et consumer lag Kafka normal < 30 s.

HPA pour HTTP/gRPC, KEDA pour lag Kafka/queues, Cluster Autoscaler uniquement si le provider permet le provisioning automatique.

14. CI/CD

```
Developer
-> Git
-> Tekton
-> gofmt/vet/lint
-> tests + race
-> build OCI
-> Trivy
-> Syft SBOM
-> Cosign
-> Harbor
-> GitOps repo
-> FluxCD
-> Flagger canary
-> Kubernetes
```

Les images de production sont referencees par digest SHA256. Harbor utilise des Robot Accounts a privileges minimaux.

15. Progressive delivery

Configurer Flagger avec Istio et Prometheus pour les services critiques. Rollback automatique sur degradation des metriques. Les services simples peuvent rester en RollingUpdate.

Flip/OpenFeature reste distinct : feature flag applicatif, pas mecanisme de deploiement.

16. Backup et PRA

PostgreSQL -> WAL + full backup + PITR
OpenBao -> Raft snapshots
Kubernetes -> Velero + CSI snapshots
Object -> versioning + immutability + inter-site backup/replication
Git -> immutable backup
Config -> Git + FluxCD

Automatiser des tests de restauration. Prevoir un exercice PRA complet inter-site periodique.

Le failover des ecritures respecte home_site, fencing/quorum et prevention du split-brain. Le DNS/GSLB ne suffit jamais a autoriser une promotion d'ecriture.

17. Developer Platform

Backstage est l'UX principale. La Platform API est le contrat de plateforme et la CLI sert aux developpeurs/automatisations.

Developer
-> Backstage / CLI
-> Platform API
-> OPA authorization
-> declarative change
-> PR GitOps
-> CI/policies
-> FluxCD

Platform API : idempotence, operations longues asynchrones, versioning N/N-1, audit request_id/change_id/actor/PR/result.

18. Diagnostic

KubeBuddy et K8sGPT peuvent etre utilises en lecture seule pour acclereler le diagnostic. Ils ne corrigent jamais automatiquement la production.

diagnostic -> proposition -> PR -> CI/policies -> FluxCD -> remediation

19. Tests de resilience

Integrer k6 et Chaos Mesh. Tester au minimum : perte worker, broker Kafka, failover PostgreSQL, coupure WAN, panne Site A, panne provider externe, saturation queue et rollback Flagger.

Mesurer RTO/RPO reels plutot que les supposer.

20. Tests de securite

Verifier NetworkPolicies default-deny, egress allowlist, Pod Security restricted, rotation credentials, revocation IAM, KMS/encryption-at-rest, signatures d'images, policies Kyverno, isolation namespaces et procedure break-glass.

21. Ordre d'implementation

1. Repo IaC/GitOps + governance
2. Terraform + Ansible baseline Site A
3. Kubernetes + Cilium
4. FluxCD
5. OpenBao + Keycloak/SPIRE integration
6. Istio Ambient + Kong + edge
7. Storage
8. PostgreSQL/Kafka/RabbitMQ/Redis/OpenSearch
9. Observability
10. Tekton/Harbor/Flagger
11. Backstage + Platform API
12. Backup/restore
13. Chaos/performance/security tests
14. Reproduire Site B
15. Mirror/failover multi-site

22. Definition of Done

- Un cluster Site A peut etre reconstruit depuis IaC/Git.
- Un microservice est deploye de bout en bout par Tekton -> Harbor -> FluxCD.
- Les secrets runtime proviennent d'OpenBao et les identites sont individuelles/workload.
- Le trafic ingress, service-to-service et egress respecte les policies.
- Les workloads critiques survivent a une maintenance de noeud.
- Les backups PostgreSQL/OpenBao/Kubernetes/Git sont restaurables.
- Site B est reproductible depuis la meme base declarative.
- Le failover multi-site est teste sans split-brain.
- Les SLO, logs, traces, metrics et audit sont visibles.
- Un test Chaos/PRA produit des preuves et un rapport exploitable.

Le livrable est termine uniquement lorsque la plateforme est reconstructible, observable et restauree depuis backup lors d'un test. Un cluster qui fonctionne mais que l'on ne sait pas reconstruire n'est pas considere termine.